

README.md

Video implementation:

https://drive.google.com/drive/folders/1fbmC_Ca_dbldugXSeg5V8kE76YbiRS1F?usp=drive_link

fytz: Your Personal Wardrobe & Outfit Simulator

Table of Contents

1. Description
 2. Requirements
 3. Installation & Setup
 4. Running the App
 5. Creating an Account
 6. Features Walkthrough
 - Adding Clothes to Your Closet
 - The Virtual Closet (Home Page)
 - Outfit Recommendations
 - Outfit History
 - Settings
 7. Notes & Known Limitations
-

Description

fytz is a web-based virtual wardrobe application that lets you upload, organize, and style your clothing digitally. You can take pictures of your clothing items and add them to your fytz wardrobe as either shirts/tops, dresses, pants/skirts/shorts, shoes, and sunglasses. You can then use your interactive mannequin/outfit vision board to look at different clothing combinations. You can do this by either choosing your clothes straight from your digital wardrobe or getting AI-powered outfit recommendations based on real-time weather. You can then save the outfit to your history and even choose past outfits from your history to look at on your mannequin again.

Requirements

Before running FYTZ, make sure you have the following installed on your machine/virtual workspace:

- **Python 3.10+**
- **pip** (Python package manager) → helps with Flask, Pillow, rembg, cs50, etc.

- A terminal / command prompt

The **fytz** webpage runs entirely locally which means there is no external database server or cloud service is required. However, internet access is needed for live weather data (pulled from weather.gov) to provide outfit recommendations and the Bootstrap/Font Awesome CDN links used for webpage styling.

Installation & Setup

1. Unzip the project by typing the following in your terminal:

```
Shell
unzip fytz.zip
rm -f fytz.zip
cd fytz
```

2. (Optional) Create a virtual environment

```
Shell
python3 -m venv venv
source venv/bin/activate      # macOS / Linux
venv\Scripts\activate        # Windows
```

3. Install any dependencies

```
Shell
pip install -r requirements.txt
```

This will install Flask, Flask-Session, the CS50 SQL library, Pillow, **rembg** (AI background removal), Werkzeug, and Requests. It is also important to know that **rembg** downloads a small AI model on first use, so this may take a moment the first time you upload a clothing item or photo. The zip file already contains a requirements.txt, but, just in case you can run the line of the code above.

4. Verify the database

The zipped project folder already contains `fytz.db` (a pre-configured SQLite database), so no additional setup is needed as the schema is already in place. If you ever need to reset the database from scratch, the tables are:

- `users` — stores accounts, preferences, and profile images
- `wardrobe` — stores individual clothing items per user
- `outfits` — stores saved outfit combinations with weather context

And to view the schema:

```
(venv) fytz/$ sqlite3 fytz.db
sqlite3> .schema
```

Running the App

From inside the `fytz/` directory, run:

```
Shell
flask run
```

And click the link provided. The app will be running locally on port 5000. You should see the **fytz** login screen. You don't need to download flask individually, because the `requirement.txt` or the pip install command from before should have all the required parts.

Creating an Account

For demonstration purposes and to test functionality, you can log into this account:

Username: ay

Password: abc

You can create a new account or “Join fytz.” by either clicking the **Register** in the top navigation bar or clicking the Register button on the Login page where it says “Don't have an account?”. You enter a new username and a password (which is protected but you can use the eye icon next to each password field to toggle visibility while typing), confirm the password, and click **CREATE ACCOUNT**. Usernames must be unique, and if the username is already taken, you will see an error message stating “Username already exists”. After registering, you are automatically logged in and redirected to your (empty) fytz closet.

Once you have created your account, you are saved as a new user with a unique `user_id`, and so when you want to login again, you can simply visit the “Log In” page and type in your username and password, click **LOG IN**, and you will be taken to your fytz closet.

Features Walkthrough

Adding Clothes to Your Closet

Navigate to your **Closet** via the top navigation bar where you will see a button **Add Item** and once you click that button you will be taken to the **Upload** page. You will see a form where you can do the following:

1. **Choose a category for the clothing you are about to upload** — Shirts/tops, Pants/skirts/shorts, Shoes, or Shades(Sunglasses).
2. **Optionally add tags** — e.g., "jeans, casual, blue" — to help the recommendation engine find relevant items and you can also search through your closet based on these tags
3. **And finally, upload an image** of the clothing item (JPG, PNG, or similar formats).

! MAKE SURE THE FILE NAME IS DESCRIPTIVE (ex. Your formal dress pants should be named formal_pants). This helps the outfit recommender understand what to recommend based on your STYLE VIBE (Casual, Formal, or Sporty) and also current temperature !

When you click the **save** button, **ftyz** uses an AI background removal tool (**rembg**) to strip the background from your photo, leaving just the clothing item on a transparent background. This is what makes items look clean when being used on the mannequin and outfit vision board. The processed image is stored in **static/uploads/** and the file's name is saved in the database with the associated user's `user_id`.

★**Tip:** For best results, photograph clothing on a plain, contrasting background (e.g., a white wall). The AI model handles most backgrounds well, but complex or cluttered backgrounds may produce imperfect cutouts. ★

If you decide you no longer want to add an item you can always hit the **cancel** button at the very bottom of the upload panel and you will be taken back to the homepage/closet.

Home Page: The Virtual Closet & Outfit Simulator

The home page is the heart of **ftyz**. It consists of three main panels:

Left panel — Your Closet: Your uploaded items are displayed here in a tabbed grid: All, Shirts, Pants, Shoes, and Shades. The items are shown on multiple pages depending on how many

items you have, you can toggle through the pages using the buttons at the bottom of the panel. You can:

- **Click any item** and it will be added to the right panel that serves as an outfit vision board and a mannequin of sorts.
- **Click on the categories** to only show items of that certain category (ex. Click on **Shoes** to only see pairs of shoes)
- **Click the heart icon** (♥) on a card to mark an item as a favorite (turns ♥) and you can toggle to only show favorited items.
- **Use the search bar** to filter items by tag keywords (e.g., searching "jeans" will show pants tagged with "jeans" by the user, however, there is no AI/automatic tagging).
- **Select multiple items** using the checkboxes, and either use the **Select All** button (to select all the items) or the **Delete Selected** button to delete the selected items in bulk.

Right panel — The Mannequin/Outfit Vision Board: A virtual mannequin displays your selected outfit in real time. The panel is interactive, so you can move each piece of clothing by grabbing, moving, and clicking to place. Each clothing category layers independently and you can have one of each clothing category on your mannequin at all times, so you can mix and match freely. Once you're happy with a look:

- Click **finalize fit.** to save the outfit to your history along with the current weather data (this log can be used later to re-wear the same outfit)
- Click **clear** to reset the mannequin and remove all the items from the mannequin
- Grab a piece of clothing and drag it to the bottom right of the panel to the "🗑️" icon to remove that item from the mannequin.
- Grab each piece of clothing and reposition them for a better visual fit.

Weather Banner A weather strip in the navigation bar shows your current temperature, forecast, and any active weather alerts. This is pulled live from the National Weather Service and cached in your browser for 30 minutes to avoid unnecessarily frequent API calls.

Outfit Recommendations

Click the **"WHAT SHOULD I WEAR?"** button (magic wand icon ✨) in the navigation bar in the top left corner at any time. fytz will automatically pick one item from each category — filtered by the current temperature and your saved style preference (as indicated by the weather API and the settings you set) and dress the mannequin with the suggested combination.

Also, you can toggle **"Favorites Only"** on the home page to limit recommendations to items you have hearted. Or you can use the search bar to search for specific tags you added to the items (ex. Search "summer" and the sundress you tagged with "summer" will show in your closet when you are on the shirts/tops or all tab).

The fytz recommendation follows this logic and filters items by:

- **Temperature** — cold weather (below 45°F) → jackets, boots, and jeans; hot weather (above 90°F) → shorts, sandals, and shades, normal weather (between 45°F and 90°F) → random (mainly based on style preference)
 - **Style Vibe** — Formal → suits, dress shirts, and boots; Sporty → gym wear, sneakers, and spandex; Casual → jeans, t-shirt, and sandals.
-

Outfit History

Navigate to **History** using the nav bar at the top of the page to see every outfit you have saved, displayed in reverse chronological order. Each card shows:

- Thumbnail images of the mannequin with the shirt, pants, shoes, and shades worn.
 - The temperature and weather forecast at the time the outfit was saved.
 - A heart button to favorite an outfit in your History.
 - The cards are clickable, so when you click on the card it sends the outfit directly back to the home page mannequin so you can “re-wear” it.
 - A **delete log** button to permanently remove the history entry.
-

Settings

Navigate to **Settings** (</settings>) to manage your account and personalize FYTZ.

Profile Picture — Upload a photo of your face. FYTZ will use it as the profile picture for the profile image in the top right corner at all times, and also as your profile picture on the settings page. You can optionally **Remove current photo** to reset your profile image back to the default profile image.

Mannequin Head — Separate from your profile picture, you can upload a specific headshot for the mannequin. This replaces the generic mannequin head with your likeness (the generic mannequin head is based on gender).

Personal Info — Set your gender (used for default mannequin head selection), your home address (used to localize weather data), and your style preference (Casual, Formal, or Sporty) which dictates what fytz will recommend for an outfit).

Username & Password — Update your username or change your password. Changing your password requires entering your current password first. You can also use the eye icon to view the password fields as necessary.

Wardrobe Stats — At the bottom of Settings, you can see a summary of how many items you have in your closet, how many outfits you have saved, and a breakdown by category.

Notes & Limitations

- **Weather is US-only.** The app uses the National Weather Service API (api.weather.gov), which only covers US locations. If you set a non-US address, the app will fall back to Cambridge, MA weather data.
- **Background removal takes a few seconds.** The first upload after starting the server will be slowest because [rembg](#) loads its AI model into memory. Subsequent uploads are faster.
- **Image formats.** fytz processes all uploads through Pillow and saves them as [.png](#) with transparency. The original uploaded file format does not matter.
- **Deleting wardrobe items** uses a soft/hard delete strategy. If an item has been worn in a saved outfit, it is only hidden from your closet (soft delete) so that your history remains accurate. Items that have never been worn are permanently deleted along with their image file.
- **Sessions are stored on the filesystem** in the [flask_session/](#) folder. Clearing this folder will log out all active users.

DESIGN.md

fytz — Design

This document provides a technical walkthrough of how FYTZ is built, the decisions behind the architecture, and the tradeoffs considered along the way.

Overview & Stack

FYTZ is a multi-user web application built with **Flask** (Python) on the back end and vanilla **JavaScript** with **Bootstrap 5** on the front end. Data is persisted in a **SQLite** database accessed through the **CS50 SQL library**, which wraps SQLite with a clean parameterized-query interface and protects against SQL injection by default.

The decision to use Flask over a heavier framework like Django was deliberate: FYTZ is a focused single-user-per-session app with a modest number of routes, and Flask's micro-framework approach kept the codebase lean without sacrificing flexibility. SQLite was chosen because it requires zero infrastructure — no database server to install, configure, or run — which makes local setup and submission trivial. For an application at this scale (one file, one user at a time), SQLite is entirely appropriate.

Database Schema

The database has three main tables.

users stores account credentials and all per-user preferences in a single row. The **hash** column stores a Werkzeug-generated bcrypt hash of the password — plaintext passwords are never stored. Optional columns (**profile_image**, **gender**, **address**, **style_preference**, **mannequin_head**, **registered_at**) start as **NULL** and are filled in as users configure their profiles. Keeping all user metadata in one table avoids joins for the common case of loading a user's profile.

wardrobe stores individual clothing items. Each row belongs to a **user_id**, has a **category** (shirts, dresses, pants, shoes, or shades), a **tags** text field for search, a filename pointing to the processed image in **static/uploads/**, and two boolean integer flags: **is_favorite** and **is_deleted**. The **is_deleted** flag is the key to the soft/hard delete strategy described later.

outfits stores saved outfit combinations. Each row holds four nullable foreign keys pointing to wardrobe items (**shirt_id**, **pants_id**, **shoes_id**, **shades_id**), a **date_worn** timestamp

(defaulting to `CURRENT_TIMESTAMP` so the database handles it automatically), and weather columns (`temp_worn`, `forecast_worn`, `icon_worn`) that snapshot the conditions at the time the outfit was saved. Using `LEFT JOIN` when querying history means that even if a referenced wardrobe item is later deleted, the outfit history row still exists and renders gracefully with a missing image rather than crashing.

Authentication & Sessions

Authentication is handled manually using Werkzeug's `generate_password_hash` and `check_password_hash`. Flask's server-side session (via `flask-session` with filesystem storage) stores only the `user_id` integer after a successful login. Every protected route is decorated with a custom `@login_required` decorator that checks `session.get("user_id")` and redirects to `/login` if it's absent.

A `@app.context_processor` called `inject_user` runs on every request and injects a `user` dictionary (fetched from the database) into every template's context. This means Jinja2 templates can reference `{{ user.username }}` or `{{ user.profile_image }}` everywhere without each route handler needing to pass user data explicitly. This was a deliberate DRY (Don't Repeat Yourself) choice — without it, roughly a dozen route handlers would each need their own `db.execute("SELECT * FROM users...")` call.

Image Processing Pipeline

Every clothing image and profile photo goes through an AI background removal step powered by `rembg`, which uses a pretrained U-2-Net deep learning model to separate the foreground subject from the background. The output is a PNG with full alpha-channel transparency.

This was the most impactful design decision in the entire project. Without background removal, layering clothing images on the mannequin would look visually messy — each image would arrive with its own white or cluttered background, making the outfit canvas look like a collage of rectangles. With transparent PNGs, CSS `position: absolute` stacking creates a clean, layered try-on effect that feels natural.

The tradeoff is speed: `rembg` loads a ~170MB neural network model into memory on first use. Subsequent calls are fast because the model stays resident. To handle the latency gracefully, the upload route processes the image synchronously and redirects only after completion, so the user always sees the finished result rather than a partially processed image. An async job queue (like Celery) was considered but deemed unnecessary for a single-user app where one upload at a time is the realistic use case.

Profile pictures go through the same pipeline with an optional background-removal toggle, since some users may want to upload a photo that's already clean (e.g., a headshot on a white background) without re-processing it.

Front-End Architecture

The front end is split into two JavaScript files:

main.js handles everything that is page-agnostic: weather fetching and display, the outfit recommendation trigger, password toggle visibility, outfit history interactions (favorite, delete, reload), and the settings page's change-tracking UI (the save/cancel button logic that appears only when a form field has been modified).

mannequin.js handles everything specific to the home page closet: loading wardrobe items via `/api/wardrobe`, rendering item cards into tabbed panels, the drag-and-drop mannequin positioning system, item selection and deselection, tag-based search filtering, bulk delete, and the "Finalize Fit" call to `/finalize`.

The separation keeps each file focused. **mannequin.js** is conditionally loaded only when `request.path == "/"` in `layout.html`, so it does not execute on Settings, History, or Upload pages where its logic would be irrelevant.

Mannequin Slot System & Initial Positioning

When a user clicks a wardrobe card, `selectItem()` is called with the item's category, image filename, and database ID. The mannequin canvas is divided into four invisible `<div>` slots — `slot-shirts`, `slot-dresses`, `slot-pants`, `slot-shoes`, `slot-shades` — each positioned absolutely inside the mannequin container. `selectItem()` injects a `<img class="draggable-item">` into the correct slot and sets a category-appropriate starting size and `top` offset:

- **Shades** — `width: 55px, top: 35px` (face level)
- **Shirts** — `width: 150px, top: 80px` (torso)
- **Pants** — `width: 130px, top: 180px` (lower body)
- **Shoes** — `width: 100px, top: 310px` (feet)
- etc.

These defaults are hand-tuned to land each item roughly in the right anatomical zone on the mannequin silhouette, so the outfit looks composed without the user needing to immediately reposition anything.

Drag-and-Drop Implementation

The drag system is built entirely from native DOM mouse events — no external drag-and-drop library is used. Three event listeners work together:

mousedown — A global listener on `document` watches for clicks on any element with the class `draggable-item`. When one is found, it becomes the `activeElement`. The listener records `initialX` and `initialY` as the offset between the mouse cursor position (`e.clientX`, `e.clientY`) and the element's current `left/top` CSS values (read via `window.getComputedStyle`). This offset anchoring is important: without it, the image would snap its top-left corner to the cursor on pickup, causing a jarring jump. It also immediately promotes the slot's `z-index` to `9999` so the item being dragged floats above all other layers during the drag gesture.

mousemove (drag()) — Registered dynamically on `document` only while a drag is active. On every mouse move event, it computes the new position as `e.clientX - initialX` and `e.clientY - initialY`, and applies these directly to `activeElement.style.left` and `activeElement.style.top`. It simultaneously checks whether the cursor is hovering over the trash bin by comparing `e.clientX/e.clientY` against the bin's `getBoundingClientRect()`. If the cursor enters the bin, the bin gets the `trash-active` CSS class (a visual highlight); leaving removes it.

mouseup (stopDrag()) — Finalizes the drag. If the cursor is over the trash bin at release, the item's slot is cleared, `selectedOutfitIds[category]` is set back to `null`, and a crunch sound effect plays. Otherwise, the item stays at its dropped position, with a boundary clamp that prevents it from being dragged above `top: 0` or beyond the right edge of the mannequin box (using `getBoundingClientRect()` of the container vs. the element's `offsetWidth`). Both the `mousemove` and `mouseup` listeners are removed from `document` immediately after the drag ends, so they do not continue firing unnecessarily.

Z-Index Layering

The four clothing slots have a fixed z-index hierarchy that reflects how clothes layer on a real body. This is enforced in `stopDrag()`: when the user releases the mouse, the slot's z-index is reset to its category's fixed value from the `zLayers` lookup object. During a drag, the slot is temporarily promoted to `9999` (set in the `mousedown` handler) so the item being moved always renders on top of everything else, including other already-placed clothing. As soon as the drag ends, it snaps back to its anatomically correct layer. The result is that, in the resting state, shades always render on top of shirts (as they would in real life), shirts over pants, and pants over shoes, regardless of what order the user dressed the mannequin.

The wardrobe data is loaded on page load via a single `fetch('/api/wardrobe')` call that returns all of the user's non-deleted items as JSON. This was chosen over server-side rendering of wardrobe cards (e.g., passing items via Jinja2) because the mannequin panel and closet panel need to share the same JavaScript state. Specifically, `selectedOutfitIds`, which

tracks which item is currently dressed on each mannequin slot. Having the data in a JavaScript array (`allItems`) also makes client-side tag search instant without a round-trip to the server.

Weather Integration

Weather data is fetched from the **National Weather Service API** (`api.weather.gov`), a free, no-key-required public API. The flow has two steps: first, the app queries `/points/{lat},{lon}` to get the user's NWS grid metadata and the URL for their hourly forecast; then it fetches that forecast URL for the current period's temperature and condition. Active weather alerts are fetched separately from `/alerts/active?point={lat},{lon}`.

If the user has set a home address in Settings, the app uses **OpenStreetMap Nominatim** to geocode the address to lat/lon coordinates before the NWS call. If no address is set, the app defaults to Cambridge, MA (lat 42.3736, lon -71.1097).

On the front end, weather data is cached in `localStorage` for 30 minutes. The cache key `weather_timestamp` is checked on every page load; if fewer than 30 minutes have elapsed, the saved JSON is used instead of making a new API call. The cache is invalidated by setting the timestamp to `'0'`, which the `DOMContentLoaded` logic treats as expired. This avoids hammering the NWS API on every navigation between pages and keeps the navbar weather display snappy.

The weather data is also written to the Flask session (`session["weather_data"]`) at fetch time. This is what the `/finalize` route reads when saving an outfit — since the outfit save happens client-side (a `fetch` POST to `/finalize`), the server needs to have access to the weather data that was last displayed to the user. Storing it in the session bridges the gap between what JavaScript knows and what the database needs.

Outfit Recommendation Engine

The recommendation system at `/api/recommend` is a rule-based filter built with dynamic SQL query construction. Rather than using a machine learning model, the engine assembles a `SELECT` query with `WHERE` clauses that filter by temperature range and style preference using `LIKE` matches on the `image_file` column.

The filename-based matching approach was chosen because it leverages the naming convention that naturally emerges when users upload files — filenames like `navy_blue_suit.png`, `sneakers.png`, or `black_jeans.png` carry semantic information. While this is imperfect (a file named `img_20250501.png` would not match anything), the tag

system provides a secondary signal. When a match is found via the temperature/style filter, that item is returned; if no match is found and `favorites_only` is off, the engine falls back to a random item from that category to ensure the mannequin is always fully dressed.

One item is selected per category using `ORDER BY RANDOM() LIMIT 1`, which gives variety across recommendation calls. The query is parameterized throughout to prevent injection.

Soft vs. Hard Delete

When a user deletes a wardrobe item, the app checks whether that item has ever appeared in an `outfits` row. If it has, a **soft delete** is performed: `is_deleted` is set to `1` and the item disappears from the closet UI, but the database row is preserved. This ensures that the History page can still render old outfits accurately using the existing `LEFT JOIN` query — the image file is retained on disk.

If an item has never been worn (no matching `outfits` row), a **hard delete** is performed: the database row is removed and the corresponding file in `static/uploads/` is deleted from disk using `os.remove()`. This keeps the server storage clean and prevents orphaned image files from accumulating.

This dual strategy means the app never loses historical accuracy while also not hoarding files for clothes that were never used.

Settings Page UI Pattern

The Settings page uses a progressive disclosure pattern for its forms: every editable section starts with the save button disabled and a cancel button hidden. JavaScript's `trackChange()` function listens for `oninput` events on every field in a form. When any field's current value differs from its `defaultValue` (the value baked into the HTML at page load), the save button is enabled and the cancel button appears. `cancelChange()` resets all fields to their `defaultValue` and hides the buttons again.

This pattern prevents accidental saves (the button is grayed out unless something actually changed) and gives users a clear escape hatch without a full page reload. The implementation deliberately avoids any framework — it is pure DOM manipulation with `querySelector`, `classList`, and the native `defaultValue` / `defaultChecked` properties, which are set automatically by the browser when the HTML is parsed.

Template Inheritance

All pages extend `layout.html` via Jinja2's `{% extends %}` / `{% block %}` system. The layout handles the navbar (including weather display, profile avatar, and the "What to wear?" AI button), flash message display, Bootstrap and Font Awesome CDN links, and script loading. Individual page templates only define the `{% block main %}` content and their `{% block title %}`.

This structure means changes to the nav, footer, or global scripts propagate to every page automatically. The navbar conditionally renders the weather strip, nav links, and profile icon only when `session["user_id"]` is set, so the login and register pages see a clean, minimal header.